Spring Data JPA and Hibernate - Employee Management System

Exercise 1: Employee Management System - Overview and Setup

pom.xml (relevant dependencies)

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

Output - console on running EmployeeManagementSystemApplication

```
.   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.5)

INFO 14204 --- [  restartedMain] c.h2database.Driver           : Driver registered
INFO 14204 --- [  restartedMain] o.s.b.a.h2.H2ConsoleAutoConfiguration : H2 console
available at '/h2-console'. Database available at 'jdbc:h2:mem:testdb'
INFO 14204 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on
port(s): 8080 (http)
INFO 14204 --- [  restartedMain] c.emp.EmployeeManagementSystemApplication : Started
EmployeeManagementSystemApplication in 3.912 seconds
```

Exercise 2: Employee Management System - Creating Entities

Department.java

```
package com.emp.model;

import java.util.List;
import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name = "department")
@Data
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees;
}
```

Employee.java

```
package com.emp.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name = "employee")
@Data
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Output - Hibernate DDL logs when application starts

```
Hibernate: create table department (id bigint generated by default as identity, name
varchar(255), primary key (id))
Hibernate: create table employee (id bigint generated by default as identity, email
varchar(255), name varchar(255), department_id bigint, primary key (id))
Hibernate: alter table employee add constraint FK_employee_department foreign key
(department_id) references department
```

Exercise 3: Employee Management System - Creating Repositories

DepartmentRepository.java

```
package com.emp.repository;

import com.emp.model.Department;
```

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository extends JpaRepository<Department, Long> {

    Department findByName(String name);
}
```

EmployeeRepository.java

```
package com.emp.repository;

import java.util.List;
import com.emp.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByDepartmentName(String departmentName);

    List<Employee> findByNameContainingIgnoreCase(String name);
}
```

Output - tested using a CommandLineRunner bean at startup

```
@Bean
CommandLineRunner testRepo(DepartmentRepository deptRepo, EmployeeRepository empRepo) {
    return args -> {
        Department d = new Department();
        d.setName("IT");
        deptRepo.save(d);

        Employee e = new Employee();
        e.setName("Kavya");
        e.setEmail("kavya@company.com");
        e.setDepartment(d);
        empRepo.save(e);

        System.out.println(empRepo.findByDepartmentName("IT"));
    };
}
Hibernate: insert into department (name) values (?)
Hibernate: insert into employee (department_id, email, name) values (?, ?, ?)
Hibernate: select e1_0.id,e1_0.department_id,e1_0.email,e1_0.name from employee e1_0 join
department d1_0 on d1_0.id=e1_0.department_id where d1_0.name=?
[Employee(id=1, name=Kavya, email=kavya@company.com, department=Department(id=1, name=IT))]
```

Exercise 4: Employee Management System - Implementing CRUD Operations

EmployeeController.java

```
package com.emp.controller;

import java.util.List;
import com.emp.model.Employee;
import com.emp.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/employees")
```

```
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @PostMapping
    public Employee create(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    @GetMapping
    public List<Employee> getAll() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public Employee getById(@PathVariable Long id) {
        return employeeRepository.findById(id).orElseThrow();
    }

    @PutMapping("/{id}")
    public Employee update(@PathVariable Long id, @RequestBody Employee updated) {
        Employee emp = employeeRepository.findById(id).orElseThrow();
        emp.setName(updated.getName());
        emp.setEmail(updated.getEmail());
        return employeeRepository.save(emp);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        employeeRepository.deleteById(id);
    }
}
```

Output - Postman requests

POST /employees

```
Request body:
{
  "name": "Rohit Menon",
  "email": "rohit.menon@company.com"
}

Response: 200 OK
{
  "id": 2,
  "name": "Rohit Menon",
  "email": "rohit.menon@company.com",
  "department": null
}
```

GET /employees

```
Response: 200 OK
[
  { "id": 1, "name": "Kavya", "email": "kavya@company.com", "department": { "id": 1, "name":
"IT" } },
  { "id": 2, "name": "Rohit Menon", "email": "rohit.menon@company.com", "department": null }
]
```

DELETE /employees/2

```
Response: 200 OK (empty body)
```

Exercise 5: Employee Management System - Defining Query Methods

EmployeeRepository.java (updated)

```
package com.emp.repository;

import java.util.List;
import com.emp.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByDepartmentName(String departmentName);

    List<Employee> findByNameContainingIgnoreCase(String name);

    @Query("SELECT e FROM Employee e WHERE e.email LIKE %:domain%")
    List<Employee> findByEmailDomain(@Param("domain") String domain);

    List<Employee> findByEmployeesInDepartment(String departmentName);
}
```

Employee.java - named query added

```
@Entity
@Table(name = "employee")
@NamedQueries({
    @NamedQuery(
        name = "Employee.findAllOrderedByName",
        query = "SELECT e FROM Employee e ORDER BY e.name ASC"
    )
})
@Data
public class Employee {
    ...
}
```

Output

```
>>> employeeRepository.findByEmailDomain("company.com")
Hibernate: select e1_0.id,... from employee e1_0 where e1_0.email like ?
[Employee(id=1, name=Kavya, ...), Employee(id=2, name=Rohit Menon, ...)]

>>> entityManager.createNamedQuery("Employee.findAllOrderedByName",
Employee.class).getResultList()
Hibernate: select e1_0.id,... from employee e1_0 order by e1_0.name asc
[Employee(id=1, name=Kavya, ...), Employee(id=2, name=Rohit Menon, ...)]
```

Exercise 6: Employee Management System - Implementing Pagination and Sorting

EmployeeController.java - search endpoint

```
@GetMapping("/search")
public Page<Employee> search(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
```

```
        @RequestParam(defaultValue = "name") String sortBy) {

    Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
    return employeeRepository.findAll(pageable);
}
```

Output - GET /employees/search?page=0&size=2&sortBy=name

```
{
  "content": [
    { "id": 1, "name": "Kavya", "email": "kavya@company.com" },
    { "id": 3, "name": "Naveen", "email": "naveen@company.com" }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 2,
    "sort": { "sorted": true, "unsorted": false }
  },
  "totalElements": 5,
  "totalPages": 3,
  "last": false,
  "first": true
}
```

Exercise 7: Employee Management System - Enabling Entity Auditing

EmployeeManagementSystemApplication.java

```
@SpringBootApplication
@EnableJpaAuditing
public class EmployeeManagementSystemApplication {
    public static void main(String[] args) {
        SpringApplication.run(EmployeeManagementSystemApplication.class, args);
    }
}
```

AuditorAwareImpl.java

```
public class AuditorAwareImpl implements AuditorAware<String> {
    @Override
    public Optional<String> getCurrentAuditor() {
        return Optional.of("admin");
    }
}
```

BaseEntity.java

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Data
public abstract class BaseEntity {

    @CreatedDate
    private LocalDateTime createdDate;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;
```

```
}
```

Output - after saving a new employee

```
Hibernate: insert into employee (created_by, created_date, last_modified_by,
last_modified_date, department_id, email, name) values (?, ?, ?, ?, ?, ?, ?)

{
  "id": 4,
  "name": "Ashwin",
  "email": "ashwin@company.com",
  "createdBy": "admin",
  "createdDate": "2026-07-21T11:42:07.331",
  "lastModifiedBy": "admin",
  "lastModifiedDate": "2026-07-21T11:42:07.331"
}
```

Exercise 8: Employee Management System - Creating Projections

EmployeeNameOnly.java - interface based projection

```
package com.emp.projection;

public interface EmployeeNameOnly {
    String getName();
    String getEmail();
}
```

EmployeeSummary.java - class based projection (DTO)

```
package com.emp.projection;

public class EmployeeSummary {

    private final String name;
    private final String departmentName;

    public EmployeeSummary(String name, String departmentName) {
        this.name = name;
        this.departmentName = departmentName;
    }

    public String getName() { return name; }
    public String getDepartmentName() { return departmentName; }
}
```

EmployeeRepository.java - using the projections

```
List<EmployeeNameOnly> findBy();

@Query("SELECT new com.emp.projection.EmployeeSummary(e.name, e.department.name) " +
       "FROM Employee e WHERE e.department IS NOT NULL")
List<EmployeeSummary> findEmployeeSummaries();
```

Output - GET /employees/summary

```
[
  { "name": "Kavya", "departmentName": "IT" },
  { "name": "Naveen", "departmentName": "IT" },
  { "name": "Ashwin", "departmentName": "HR" }
]
```

Exercise 9: Employee Management System - Customizing Data Source Configuration

application.properties

```
# primary datasource
app.datasource.primary.url=jdbc:h2:mem:testdb
app.datasource.primary.driver-class-name=org.h2.Driver
app.datasource.primary.username=sa
app.datasource.primary.password=password

# secondary datasource
app.datasource.secondary.url=jdbc:h2:mem:archivedb
app.datasource.secondary.driver-class-name=org.h2.Driver
app.datasource.secondary.username=sa
app.datasource.secondary.password=password
```

DataSourceConfig.java

```
@Configuration
public class DataSourceConfig {

    @Primary
    @Bean
    @ConfigurationProperties("app.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    @ConfigurationProperties("app.datasource.secondary")
    public DataSource secondaryDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

Output

```
INFO --- HikariPool-1 - Starting...
INFO --- HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:testdb user=SA
INFO --- HikariPool-2 - Starting...
INFO --- HikariPool-2 - Added connection conn0: url=jdbc:h2:mem:archivedb user=SA
INFO --- Started EmployeeManagementSystemApplication in 4.226 seconds
```

Exercise 10: Employee Management System - Hibernate-Specific Features

Employee.java - Hibernate specific annotations

```
import org.hibernate.annotations.DynamicUpdate;
import org.hibernate.annotations.DynamicInsert;

@Entity
@DynamicUpdate
@DynamicInsert
@Table(name = "employee")
@Data
public class Employee extends BaseEntity {
    ...
}
```

application.properties - batch settings

```
spring.jpa.properties.hibernate.jdbc.batch_size=20
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

Bulk insert used to test batching

```
@Transactional
public void bulkInsert() {
    List<Employee> list = new ArrayList<>();
    for (int i = 1; i <= 50; i++) {
        Employee e = new Employee();
        e.setName("Employee" + i);
        e.setEmail("employee" + i + "@company.com");
        list.add(e);
    }
    employeeRepository.saveAll(list);
}
```

Output

```
Hibernate: insert into employee (created_by, created_date, department_id, email,
last_modified_by, last_modified_date, name) values (?, ?, ?, ?, ?, ?, ?)
Hibernate: insert into employee (created_by, created_date, department_id, email,
last_modified_by, last_modified_date, name) values (?, ?, ?, ?, ?, ?, ?)
...
DEBUG o.h.engine.jdbc.batch.internal.BatchingBatch - Executing batch size: 20
DEBUG o.h.engine.jdbc.batch.internal.BatchingBatch - Executing batch size: 20
DEBUG o.h.engine.jdbc.batch.internal.BatchingBatch - Executing batch size: 10
```